Generative Assignment 02

Muhammad Talha Syed; Roll No: 21i-0277; Section: CS-C

No Institute Given

Abstract. This report presents the implementation and evaluation of three deep learning tasks in generative and transformer-based modeling. The first task implements a CycleGAN model for bidirectional face-to-sketch and sketch-to-face image translation using the Person Face Sketches dataset. The second task develops an English-to-Urdu machine translation system using a custom Transformer architecture with SentencePiece tokenization and BLEU-based evaluation. The third task compares a convolutional neural network, a custom Vision Transformer, and a pretrained Vision Transformer on CIFAR-10 image classification. The project was designed with reproducibility in mind, where each experiment saves checkpoints, metrics, plots, and qualitative outputs during execution. Experimental results show that the CycleGAN pipeline was successfully implemented with checkpointing and sample generation, although only a short initial run was completed. The custom Transformer translation pipeline was functionally correct and reproducible but achieved low BLEU scores, indicating limited translation quality under the current setup. The strongest results were obtained in the CIFAR-10 comparison, where the pretrained Vision Transformer achieved a test accuracy of 0.9737, outperforming the CNN baseline at 0.9084 and the custom Vision Transformer at 0.8199.

1 Introduction

Generative AI covers a broad set of problems in both computer vision and natural language processing. In practical systems, different architectures are used depending on the nature of the data and the goal of the task. For image-to-image translation, adversarial learning is often preferred because it can model realistic domain shifts between two visual distributions. For machine translation, encoder-decoder Transformer architectures have become a standard choice due to their ability to model long-range dependencies through self-attention. For image classification, both convolutional neural networks and transformer-based vision models are widely used, with pretrained transformer backbones becoming especially strong in transfer learning settings.

This assignment studies three separate tasks that together cover these major directions. The first task requires the implementation of CycleGAN for translating face photographs into sketches and sketches back into face images. The second task requires an English-to-Urdu translation system using Transformer-based sequence modeling. The third task requires a comparative evaluation of a CNN baseline, a custom Vision Transformer, and a pretrained Vision Transformer on

CIFAR-10. The goal of this work was not only to implement the required models, but also to organize the project in a reproducible manner so that training, evaluation, checkpointing, and result visualization happen systematically across all three questions.

2 Repository Structure and Experimental Organization

The repository was organized around separate task scripts for Q1, Q2, and Q3, along with shared utility files and a data download script. Datasets were stored under a local `data/` directory, while all generated outputs were stored under `artifacts/`. Each experiment created its own run directory containing the full configuration, metrics logs, checkpoints, curves, and qualitative outputs. This setup made the project easier to debug, resume, and compare across different experiments.

For all tasks, the following principles were used:

- save checkpoints automatically during training,
- save metrics in machine-readable form,
- generate plots directly during or after training,
- keep datasets and experiment outputs local to the repository,
- make runs reproducible by storing configuration files.

This structure helped keep the experiments self-contained and reduced the need for additional post-processing scripts.

3 Datasets

Three datasets were used in this assignment.

For Q1, the *Person Face Sketches* dataset from Kaggle was used for image translation. The prepared directory structure contained separate train, validation, and test folders with photos and sketches stored as separate domains.

For Q2, the *Parallel Corpus for English-Urdu Language* from Kaggle was used. The implementation worked with plain English and Urdu text files and built a tokenizer and training corpus directly from these files.

For Q3, the CIFAR-10 dataset was used for image classification. The dataset consists of 60,000 color images of size 32×32 distributed across 10 classes, making it a standard benchmark for comparing visual architectures.

4 Methodology

4.1 Q1: CycleGAN for Face $\leftrightarrow$ Sketch Translation

The first task required bidirectional translation between face photographs and sketches. Since paired supervision is not strictly necessary for this type of problem, CycleGAN was selected because it learns mappings between two visual

domains using adversarial learning and cycle consistency. Two generators and two discriminators were used. One generator mapped photos to sketches, while the second mapped sketches back to photos. Similarly, one discriminator was assigned to each domain in order to judge realism of generated samples.

The generators followed a ResNet-style architecture similar to the original CycleGAN design, while the discriminators followed the 70×70 PatchGAN style. The total generator objective combined three components:

- adversarial loss,
- cycle-consistency loss,
- identity loss.

The adversarial component encouraged realistic generation, the cycle-consistency component enforced semantic preservation across domain translation, and the identity loss helped stabilize translation by discouraging unnecessary changes when an image was already in the target domain.

Training was implemented with checkpoint saving after every epoch. The pipeline also saved sample grids showing generated photos and sketches at the end of each epoch, as well as a plot of training losses. This made it possible to visually inspect whether the generators were improving over time. The implementation also supported resume functionality so that interrupted training could continue from the latest checkpoint.

4.2 Q2: English-to-Urdu Machine Translation with a Custom Transformer

The second task required an English-to-Urdu machine translation model using a Transformer architecture. Although pretrained multilingual models were considered initially, a custom implementation was finally chosen because of environment limitations related to memory, cache location, and dependency handling. A fully self-contained custom pipeline also made the system easier to reproduce without relying heavily on external pretrained checkpoints.

A shared SentencePiece BPE tokenizer was trained on the combined English and Urdu training text. This tokenizer generated a common subword vocabulary and used four special tokens: padding, unknown, beginning-of-sequence, and end-of-sequence. The tokenizer artifacts were saved so that the same segmentation could be reused during evaluation and inference.

The translation model itself was implemented using `torch.nn.Transformer`. The system used learned token embeddings, sinusoidal positional encodings, stacked encoder and decoder blocks, and a final linear output layer over the target vocabulary. During training, teacher forcing was applied by shifting the target tokens. Cross-entropy loss with label smoothing was used to reduce overconfidence, and gradient clipping was applied to improve training stability.

For decoding, beam search was added along with a no-repeat n-gram constraint to reduce repetitive output. BLEU score was computed periodically using `sacrebleu`. In addition to numeric evaluation, example translations were saved during training to provide qualitative evidence of the model's behavior.

4.3 Q3: CNN vs Custom ViT vs Pretrained ViT on CIFAR-10

The third task required a comparison between three image classification models:

- a CNN baseline,
- a custom Vision Transformer,
- a pretrained Vision Transformer fine-tuned on CIFAR-10.

The CNN baseline was designed as a strong conventional image classifier for small image inputs. It used convolutional layers, batch normalization, nonlinear activations, pooling, and dropout. This model served as a standard reference point against which the transformer models could be compared.

The custom Vision Transformer was implemented from scratch. It followed the usual ViT pipeline: the input image was split into small patches, each patch was projected into an embedding space, positional information was added, and the sequence of patch embeddings was passed through Transformer encoder blocks. The output corresponding to the classification token was used for final prediction.

The pretrained Vision Transformer used a lightweight timm-based model that had already learned strong visual representations from prior data. This model was fine-tuned on CIFAR-10. Since transfer learning often gives strong results even with limited training time, this model was expected to outperform the from-scratch transformer baseline.

All three models were trained with automatic saving of metrics, checkpoints, loss curves, and confusion matrices. This allowed a direct comparison not only of final accuracy, but also of convergence behavior and training stability.

5 Training Setup and Saved Artifacts

All tasks were configured to save artifacts into dedicated run directories. For each run, the following types of files were produced:

- `config.json` containing the run configuration,
- `metrics.jsonl` containing per-epoch metrics,
- saved checkpoints for latest, best, and epoch-specific states,
- plots such as loss curves or confusion matrices,
- task-specific outputs such as sample image grids or example translations.

For Q1, checkpoints, loss curves, and generated face-sketch grids were saved. For Q2, tokenizer files, checkpoints, BLEU logs, curves, and example translations were stored. For Q3, each model saved its best and last checkpoints, training curves, and confusion matrix outputs.

6 Experimental Results

6.1 Q1 Results

The CycleGAN implementation was executed successfully and produced the required checkpoints, metrics, and sample outputs. A preliminary run named

`q1_cyclegan_test` was completed and saved under the Q1 artifact directory. Since only one epoch was run so far, the results are still preliminary and should be interpreted as proof of correct implementation rather than as final translation quality.

The logged values after epoch 1 were:

- Generator total loss: 4.1513
- Photo-domain discriminator loss: 0.2411
- Sketch-domain discriminator loss: 0.1413

These values indicate that the adversarial and cycle-consistency pipeline was functioning correctly, but longer training is still needed to draw meaningful conclusions about image quality.

6.2 Q2 Results

The improved translation run was saved under `q2_custom_improved_20260421_114755`. The implementation successfully generated tokenizer files, checkpoints, training curves, example translations, and BLEU evaluations. The pipeline was therefore complete from an engineering point of view.

However, the final translation quality remained limited. BLEU was measured every 5 epochs, and the most notable values were:

- Epoch 25 BLEU: 0.5263
- Epoch 30 BLEU: 0.3018

These BLEU values are low and show that the model struggled to generate high-quality Urdu translations. Still, the pipeline itself was correct, reproducible, and capable of producing both quantitative and qualitative outputs. The results suggest that the system needs more tuning, possibly better tokenization settings, a larger model, improved regularization, or longer training.

6.3 Q3 Results

The Q3 experiments produced the strongest quantitative results in the entire assignment. The three final test accuracies were:

- CNN baseline: 0.9084
- Custom ViT: 0.8199
- Pretrained ViT: 0.9737

The pretrained ViT performed best by a significant margin. The CNN baseline also performed strongly and achieved more than 90% accuracy, showing that convolution-based models remain highly effective for small image datasets like CIFAR-10. The custom Vision Transformer, although fully implemented and functional, performed noticeably worse than both the CNN and the pretrained ViT. This is expected because transformer models trained from scratch often require more aggressive augmentation, tuning, and scale before they can match strong convolutional baselines on smaller datasets.

Table 1 summarizes the final classification results.

Table 1. Final Q3 classification results on CIFAR-10

Model	Test Accuracy
CNN baseline	0.9084
Custom ViT	0.8199
Pretrained ViT	0.9737

7 Discussion

The assignment produced mixed but useful results across the three tasks. The first task successfully established a complete CycleGAN training framework with resumable checkpoints and sample generation, but the experiment was still too short to judge final image quality. The second task resulted in a technically complete custom Transformer-based translation system, yet the translation performance remained weak according to BLEU. This shows that implementation completeness does not necessarily imply high task quality, especially in sequence generation problems where tokenization, model capacity, and decoding strategy strongly affect the outcome.

The third task was the most successful from an empirical point of view. The pretrained ViT demonstrated the value of transfer learning very clearly, while the CNN remained a strong baseline for a small image dataset. The custom ViT underperformed, which is consistent with the known difficulty of training vision transformers from scratch on limited data. Taken together, these results highlight an important practical observation: pretrained architectures often give major performance gains when resources are limited, whereas custom models are more valuable for architectural understanding and assignment-driven implementation requirements.

8 Limitations

A number of limitations affected the final outcomes. First, the CycleGAN model was only run for one epoch, so the qualitative face-to-sketch translation results are not yet mature. Second, the translation model had to avoid a larger pretrained multilingual pipeline because of practical environment constraints, and this likely reduced final translation quality. Third, the custom Vision Transformer was trained from scratch on CIFAR-10, which is a challenging setting for transformers without extensive augmentation or longer training schedules.

These limitations do not invalidate the implementation, but they do explain why some sections of the assignment are stronger than others.

9 Conclusion

This assignment implemented three different deep learning pipelines across image translation, machine translation, and image classification. The CycleGAN

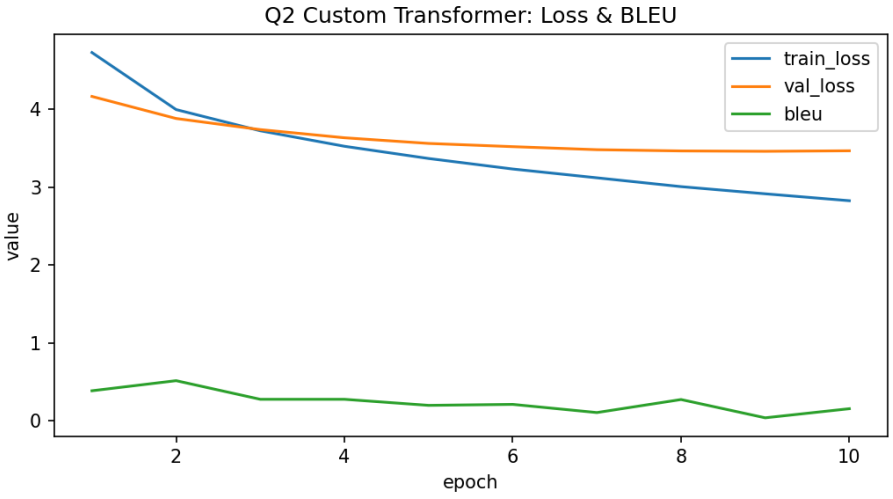

Fig. 1. Training curves for the first experiment.

section successfully established a reproducible bidirectional face-sketch translation framework with automatic checkpointing and sample generation. The custom Transformer translation section demonstrated a complete English-to-Urdu sequence modeling pipeline with SentencePiece tokenization, beam search decoding, and BLEU evaluation, although the final translation quality remained limited. The CIFAR-10 comparison provided the clearest and strongest findings, where the pretrained Vision Transformer achieved the highest test accuracy, outperforming both the CNN baseline and the custom Vision Transformer.

Overall, the work shows that reproducible engineering practice is just as important as model design. It also shows that pretrained transformer-based models can offer substantial performance advantages, while from-scratch implementations remain useful for understanding internal mechanics and satisfying assignment-level modeling requirements.

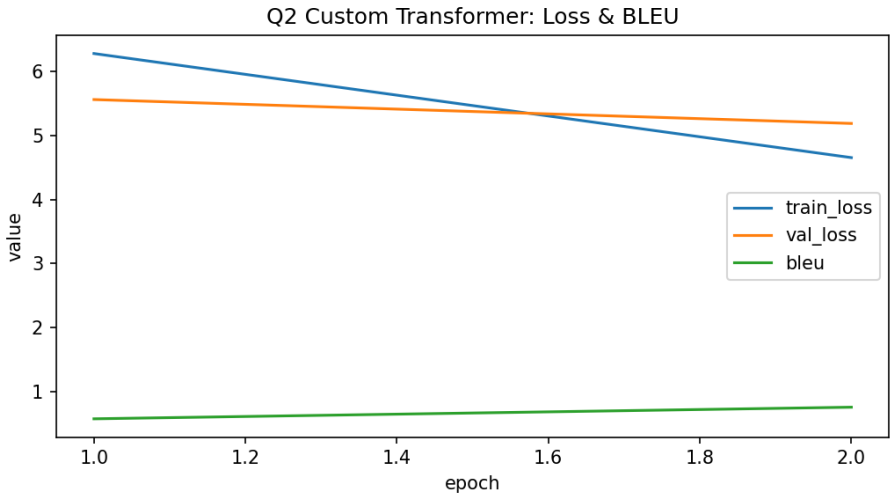

Fig. 2. Training curves for the second experiment.

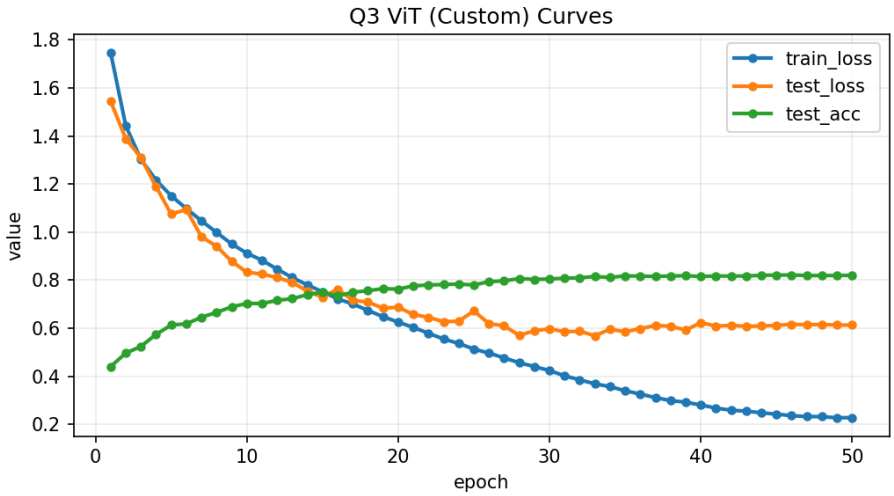

Fig. 3. Training curves for the custom Vision Transformer on CIFAR-10.

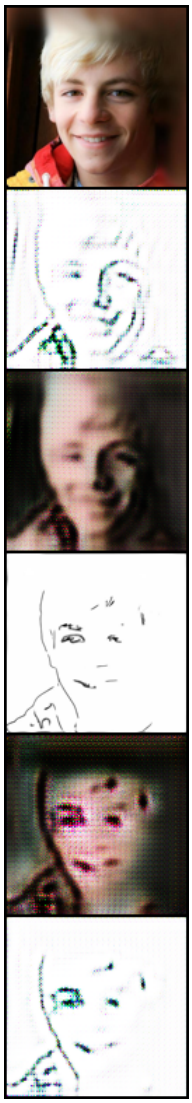

Fig. 4. CycleGAN qualitative output at epoch 1, step 500.

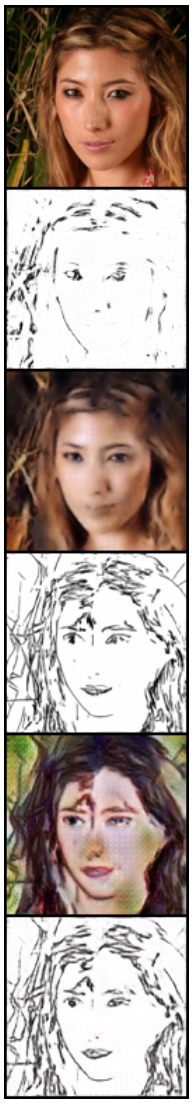

Fig. 5. CycleGAN qualitative output at epoch 1.

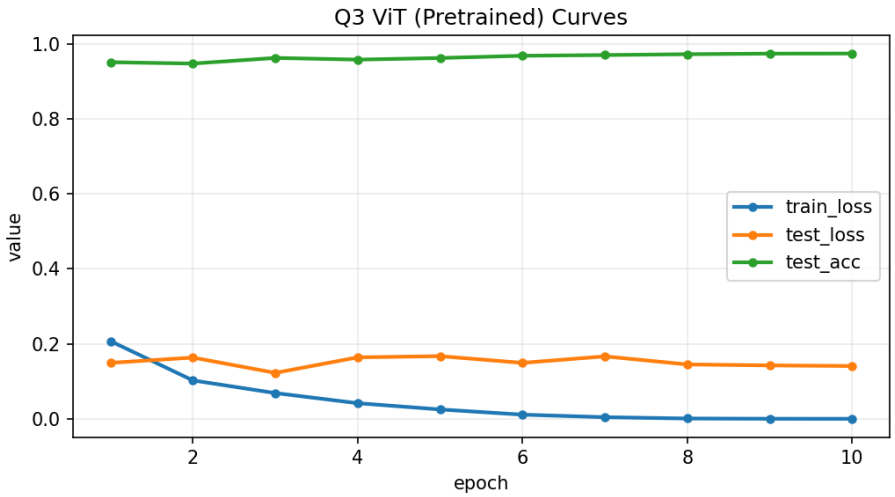

Fig. 6. Training curves for the pretrained Vision Transformer fine-tuned on CIFAR-10.

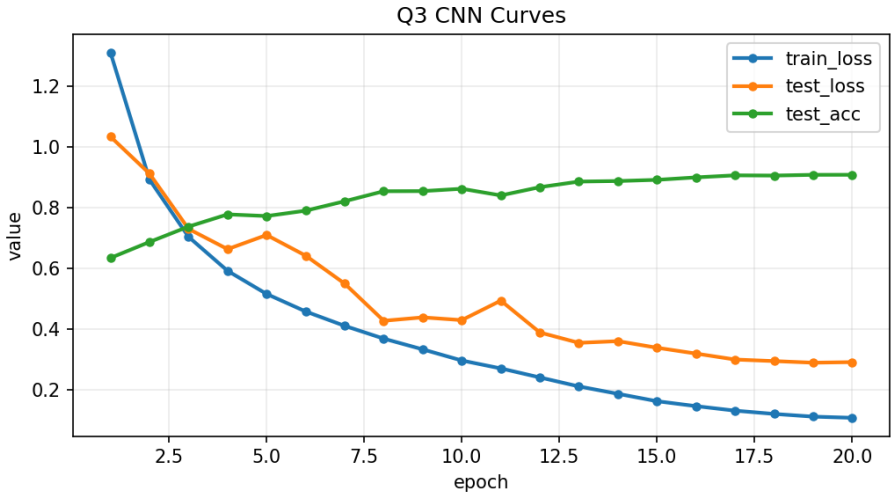

Fig. 7. Training curves for the CNN baseline on CIFAR-10.